



**ÉCOLE MAROCAINE DES
SCIENCES DE L'INGÉNIEUR**

Membre de 
HONORIS UNITED UNIVERSITIES

RAPPORT DE PROJET DE FIN DE MODULE

*Conception, implémentation, comparaison et analyse critique de modèles de deep learning pour
données tabulaires, images et séquences*

Documentation complète — Étude de faisabilité, Conception, IA & Données, Réalisation

Réalisé par:	HAITAM SALAH-EDDINE
Professeur	Mr. MOSSAB
Module	DEEP LEARNING
Filière:	IA & DATA
Groupe & Site	1 & MLY

Année universitaire 2025–2026

Table of Contents

Listes des figures	3
Listes des tableaux	4
Liste des abréviations et sigles	5
1. Introduction	7
2. Objectifs.....	7
3. Méthodologie générale.....	7
4. Partie I — MLP et données tabulaires (Wine Quality)	9
5. Partie II — CNN et vision par ordinateur (CIFAR-10)	12
6. Partie III — RNN, LSTM, GRU et Seq2Seq (Tatoeba fra-eng).....	15
7. Synthèse transversale	18
8. Limites	18
9. Conclusion.....	19
10. Annexe expérimentale	19
11. Pourquoi ces choix ? — explications simples et justifications.....	20

Listes des figures

Figure 1 - Perte et accuracy de validation selon l'initialisation.....	10
Figure 2 - Matrice de confusion (test).....	11
Figure 3 - Effet des choix architecturaux (accuracy, 5 époques).....	12
Figure 4 - Cartes de caractéristiques du premier bloc convolutionnel.....	13
Figure 5 - MLP vs CNN sur les mêmes données.	13
Figure 6 - Perplexité de validation (plus bas = mieux).	15
Figure 7 - Effet du gradient clipping (RNN simple, SGD, échelle log).....	16
Figure 8 - BLEU moyen sur la validation.....	17

Listes des tableaux

Table 1 - Performances du MLP sur l'ensemble de test (Wine Quality).....	10
Table 2 - Etude d'ablation : effet des choix architecturaux du CNN sur CIFAR-10 (accuracy de test, 5 époques).....	13
Table 3 - Comparaison MLP vs CNN sur CIFAR-10 (accuracy de test, mêmes données et mêmes époques).....	14
Table 4 - Perplexité de validation (plus bas = mieux).	16
Table 5 - Comparaison des stratégies de décodage du Seq2Seq : glouton vs beam search (BLEU moyen sur validation).....	17
Table 6 - Synthèse transversale : adéquation entre architecture et structure des données	18

Liste des abréviations et sigles

Abréviation	Terme complet / origine	Signification
Adam	Adaptive Moment Estimation	Optimiseur adaptatif utilisé pour l'entraînement.
API	Application Programming Interface	Interface de programmation (ex. API Kaggle).
BatchNorm / BN	Batch Normalization	Normalisation des activations qui stabilise et accélère l'entraînement.
BLEU	Bilingual Evaluation Understudy	Métrique de qualité d'une traduction (précision des n-grammes + pénalité de brièveté).
BOS	Begin Of Sequence	Token spécial marquant le début d'une séquence (<bos>).
BPTT	Backpropagation Through Time	Rétropropagation à travers le temps : entraînement des RNN sur la séquence dépliée.
CIFAR-10	Canadian Institute For Advanced Research (10 classes)	Jeu de 60 000 images couleur 32×32 réparties en 10 classes.
CNN	Convolutional Neural Network	Réseau de neurones convolutif, adapté aux images.
CPU	Central Processing Unit	Processeur central (ici, pas de GPU disponible).
d2l	Dive into Deep Learning	Livre/ressource de référence ; mirror public utilisé pour le corpus fra-eng.
EDA	Exploratory Data Analysis	Analyse exploratoire des données.
EOS	End Of Sequence	Token spécial marquant la fin d'une séquence (<eos>).
F1 / F1-score	—	Moyenne harmonique de la précision et du rappel.
GPU	Graphics Processing Unit	Processeur graphique, accélère l'entraînement (non disponible ici).
GRU	Gated Recurrent Unit	Cellule récurrente à portes, plus légère que le LSTM.
LFS	Large File Storage	Extension Git pour gérer les fichiers volumineux.
LSTM	Long Short-Term Memory	Cellule récurrente à portes et état de cellule, mémorise les dépendances longues.
MLP	Multi-Layer Perceptron	Perceptron multicouche : réseau de couches denses, adapté au tabulaire.
n-gramme	—	Séquence de n tokens consécutifs (utilisée par BLEU).
nn	neural network (torch.nn)	Module PyTorch contenant les couches et l'abstraction nn.Module.
PAD	Padding	Token spécial de remplissage pour aligner les longueurs (<pad>).
ReLU	Rectified Linear Unit	Fonction d'activation : $\text{ReLU}(z) = \max(0, z)$.
RNN	Recurrent Neural Network	Réseau de neurones récurrent, adapté aux séquences.

Seq2Seq	Sequence to Sequence	Architecture encodeur–décodeur pour transformer une séquence en une autre (traduction).
SGD	Stochastic Gradient Descent	Descente de gradient stochastique (optimiseur).
UCI	University of California, Irvine	Dépôt public de jeux de données (source de Wine Quality).
UNK	Unknown	Token spécial pour les mots hors vocabulaire (<unk>).

1. Introduction

Le deep learning a pour particularité de **construire automatiquement des représentations** des données à partir d'exemples. Toutefois, la performance d'un réseau ne dépend pas uniquement de sa capacité : elle dépend surtout de l'**adéquation entre son architecture et la structure statistique des données**. Ce projet illustre cette idée sur trois familles de données radicalement différentes — **tabulaire**, **image** et **séquentielle** — en concevant, entraînant et comparant les architectures correspondantes sous PyTorch.

Le travail est strictement individuel et organisé en trois parties indépendantes mais cohérentes, chacune comportant une étude théorique, une implémentation, une étude expérimentale, une analyse critique et une question de synthèse sur un jeu de données réel.

2. Objectifs

- Montrer la maîtrise de l'ingénierie PyTorch : `nn.Module`, gestion des paramètres, initialisation, sauvegarde/rechargement, utilisation correcte du device (CPU/GPU).
- Construire et comparer des architectures adaptées à chaque type de données :
 - **MLP** pour le tabulaire,
 - **CNN** (type LeNet) pour les images,
 - **RNN / LSTM / GRU / Seq2Seq** pour les séquences.
- Conduire des **expériences comparatives rigoureuses** (à conditions contrôlées) et fournir une **interprétation scientifique** des résultats.
- Répondre aux questions de synthèse de chaque partie et à la question transversale finale.

3. Méthodologie générale

3.1 Principe commun

Les trois parties partagent un même **paradigme d'apprentissage supervisé** :

1. Un modèle paramétré f_θ implémenté comme `nn.Module` ;
2. Une **propagation avant** produisant une représentation puis une prédiction ;
3. Une **fonction de perte** différentiable (entropie croisée, éventuellement masquée) ;
4. Une **rétropropagation** (autograd) calculant $\nabla_\theta L$;
5. Une **mise à jour** par un optimiseur (Adam ou SGD).

Ce qui change d'une partie à l'autre n'est pas le principe d'optimisation, mais les **hypothèses structurelles (biais inductifs)** injectées dans l'architecture.

3.2 Choix d'ingénierie transversaux

- **Reproductibilité** : graines aléatoires fixées (`seed = 42`), configuration centralisée dans une `dataclass CONFIG` par notebook (aucun chemin ni hyperparamètre codé en dur).
- **Données** : téléchargement automatique et mise en cache local (mirrors publics équivalents aux datasets Kaggle indiqués : UCI pour Wine Quality, torchvision pour CIFAR-10, mirror d2l pour fra-eng).

- **Contrainte matérielle** : environnement **CPU uniquement** (PyTorch 2.11 CPU). Les parties II et III s'entraînent donc sur un **sous-échantillon configurable** et un nombre d'époques modéré. En conséquence, **les valeurs absolues** de performance sont en deçà de l'état de l'art, mais **les comparaisons** (MLP vs CNN, RNN vs LSTM/GRU, glouton vs beam) sont menées à conditions identiques et constituent le résultat scientifique validé.

4. Partie I — MLP et données tabulaires (Wine Quality)

4.1 Cadre théorique

En PyTorch, tout réseau hérite de `nn.Module`, qui enregistre automatiquement les **paramètres** apprenables (`nn.Parameter`). Pour une couche linéaire $y = xW^T + b$, les paramètres sont les poids W et le biais b . Le **gradient** $\partial L / \partial \theta$, calculé par `loss.backward()`, mesure la sensibilité de la perte à chaque paramètre et sert à la mise à jour par l'optimiseur. Le `state_dict()` (dictionnaire {nom : tenseur}) est le format standard de **sauvegarde** des poids. Enfin, le *device* indique où résident tenseurs et modèle : données et modèle doivent **impérativement** se trouver sur le même device.

4.2 Préparation des données

Le dataset comporte 11 variables physico-chimiques et une note de qualité entière (3–8). Choix méthodologiques :

- **Cible binarisée** : *bon vin* (qualité ≥ 6) vs *ordinaire*, formulation standard pour ce jeu, bien équilibrée ($\approx 53\%$ de « bons »), évitant le bruit des classes extrêmes peu peuplées.
- **Normalisation** par `StandardScaler` **ajustée uniquement sur l'apprentissage** (pas de fuite).
- **Découpage stratifié** train / validation / test (70 / 15 / 15).

4.3 Implémentation

Le MLP est implémenté en **deux versions équivalentes** (même architecture, $\sim$ entrée $\rightarrow 64 \rightarrow 32 \rightarrow 2$ sorties) :

- Une version déclarative avec `nn.Sequential` ;
- Une version en **classe personnalisée** (`MLPCustom`) héritant de `nn.Module`.

On vérifie par `named_parameters()` que les deux versions ont **exactement le même nombre de paramètres**, confirmant leur équivalence. Le gradient, `None` avant `backward()`, apparaît après la rétropropagation.

4.4 Expériences et résultats

Initialisation. Trois stratégies sont comparées à conditions identiques : gaussienne ($\mathcal{N}(0, 0,01^2)$), constante ($W = 1$) et Xavier.

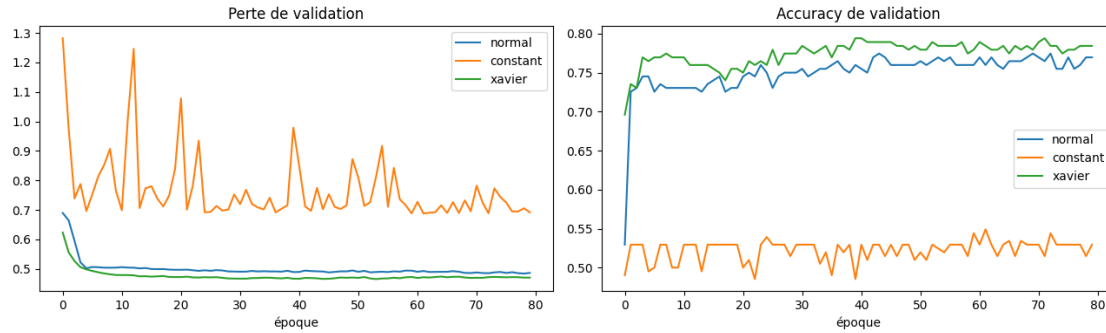


Figure 1 - Perte et accuracy de validation selon l'initialisation

Comparaison des initialisations

L'initialisation **constante** stagne : tous les neurones d'une couche étant identiques, ils reçoivent le même gradient et restent identiques (**problème de symétrie**). Les initialisations **gaussienne** et **Xavier** brisent la symétrie ; **Xavier** offre la convergence la plus stable et est donc retenue.

Sauvegarde / rechargement. Le meilleur modèle (sélectionné sur la validation) est sauvegardé via `torch.save(state_dict)` puis **rechargé** dans une architecture recréée à l'identique. On vérifie que les prédictions sont **strictement identiques** après rechargement, et que modèle et données partagent le même device.

Évaluation finale (test) :

Métrique	Valeur
Accuracy	0,726
Precision	0,789
Recall	0,657
F1-score	0,717

Table 1 - Performances du MLP sur l'ensemble de test (Wine Quality)

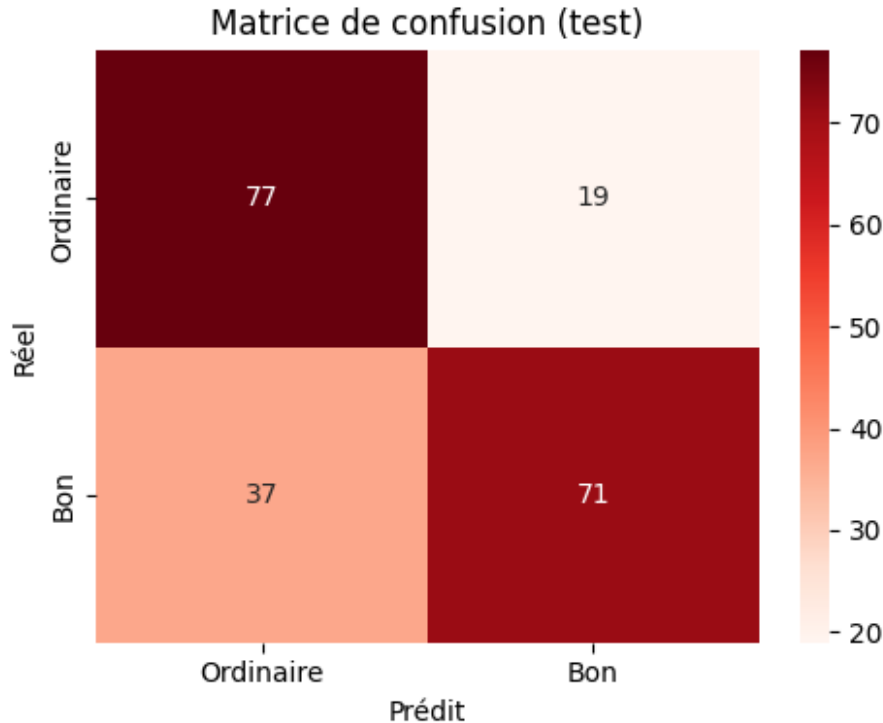


Figure 2 - Matrice de confusion (test).

Les erreurs se concentrent autour de la frontière qualité = 6, **intrinsèquement bruitée** (deux vins quasi identiques peuvent être notés 5 ou 6 selon le dégustateur).

4.5 Analyse critique

Le MLP atteint une performance honorable et **stable**, conforme à la littérature (0,72–0,78 en binaire). Ses limites tiennent à la **structure des données** : le tabulaire n'a **ni localité ni invariance** à exploiter, le biais d'un MLP n'apporte donc pas d'avantage décisif face aux modèles à arbres ; la cible est **ordinaire et bruitée** ; le **faible volume** (~1,4k lignes) accentue le risque de surapprentissage (d'où dropout et weight_decay).

4.6 Question de synthèse — Partie I

Dans quelle mesure un MLP bien paramétré constitue-t-il une solution pertinente pour la classification tabulaire, et quelles sont ses principales limites au regard de la structure statistique des données ?

Un MLP bien paramétré (normalisation, Xavier, régularisation, sélection sur validation) est un **approximateur universel** capable de modéliser des interactions non linéaires entre variables ; il fournit ici un résultat stable, ce qui valide théorie et méthodologie. Ses **limites** sont structurelles : absence de géométrie/ordre à exploiter (pas d'avantage face aux arbres), cible ordinaire bruitée plafonnant la séparabilité, et faible volume favorisant le surapprentissage. Le MLP est donc **pertinent et central pédagogiquement**, mais **pas nécessairement optimal** pour le régime tabulaire — ce qui motive, dans les parties suivantes, des architectures dont le biais inductif **correspond** à la structure des données.

5. Partie II — CNN et vision par ordinateur (CIFAR-10)

5.1 Cadre théorique

Une image CIFAR-10 fait $32 \times 32 \times 3 = 3072$ pixels. Un MLP l'aplatit, ce qui (i) **fait exploser le nombre de paramètres** et (ii) **détruit la structure spatiale**. Les CNN encodent trois biais inductifs adaptés aux images :

- **localité** (chaque neurone ne regarde qu'un voisinage),
- **partage des poids** (le même filtre glisse partout → peu de paramètres + invariance par translation),
- **hiérarchie** (bords → motifs → parties → objets).

Calculs dimensionnels. La taille de sortie d'une convolution / d'un pooling vaut $H_{out} = \lfloor (H + 2p - k) / s \rfloor + 1$. Exemples vérifiés dans le notebook : $32, k = 5, p = 0, s = 1 \rightarrow 28$; $p = 2 \rightarrow 32$ (*same padding*) ; $k = 3, p = 1, s = 2 \rightarrow 16$; $\text{max-pool } 28 \rightarrow 14$.

5.2 Implémentations « maison » vs PyTorch

La **corrélacion croisée 2D**, la **convolution multi-canaux**, le **max-pooling** et l'**average-pooling** sont réimplémentés à la main (doubles boucles) puis comparés aux couches PyTorch. Les résultats sont **numériquement identiques** (`torch.allclose` → `True`), ce qui confirme que `nn.Conv2d` réalise bien une corrélation croisée (et non une convolution à noyau retourné). Les versions PyTorch restent bien plus rapides car vectorisées.

5.3 Modèle et expériences

Le CNN est une **variante améliorée de LeNet** : blocs *Conv* → *BatchNorm* → *ReLU* → *Pooling*, suivis d'un classifieur dense. Le constructeur est **entièrement paramétré** (canaux, noyau, padding, stride, type de pooling, conv 1×1), ce qui permet de réutiliser le même code pour toutes les ablations.

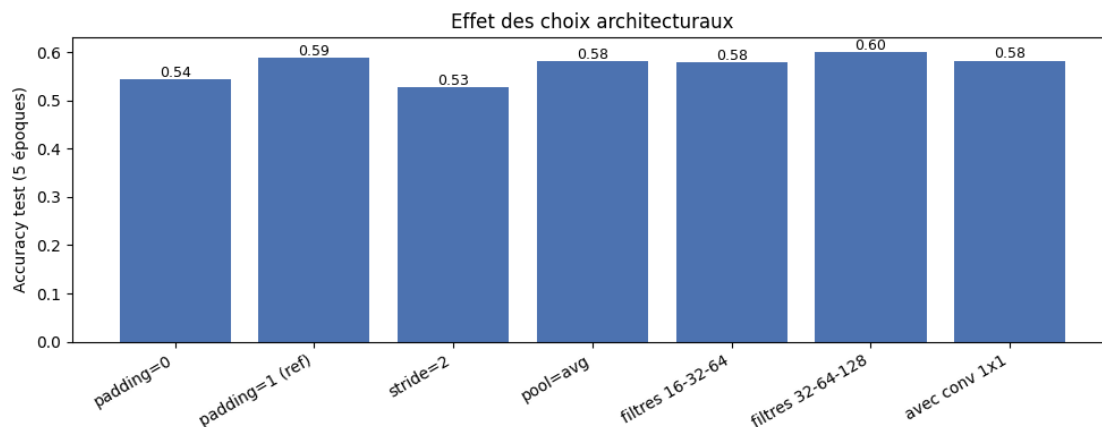


Figure 3 - Effet des choix architecturaux (accuracy, 5 époques).

On fait varier **un facteur à la fois** :

Configuration	Accuracy test
padding = 0	0,544
padding = 1 (réf.)	0,589
stride = 2	0,528
pooling = avg	0,582
filtres 16-32-64	0,578
filtres 32-64-128	0,600
avec conv 1×1	0,583

Table 2 - Etude d'ablation : effet des choix architecturaux du CNN sur CIFAR-10 (accuracy de test, 5 époques)

Lecture : conserver la résolution (padding = 1) préserve l'information de bord ; stride = 2 sous-échantillonne tôt et perd du détail ; le **max-pooling** (réf.) $\geq$ average-pooling (qui lisse) ; **plus de filtres** = plus de capacité = meilleure accuracy.

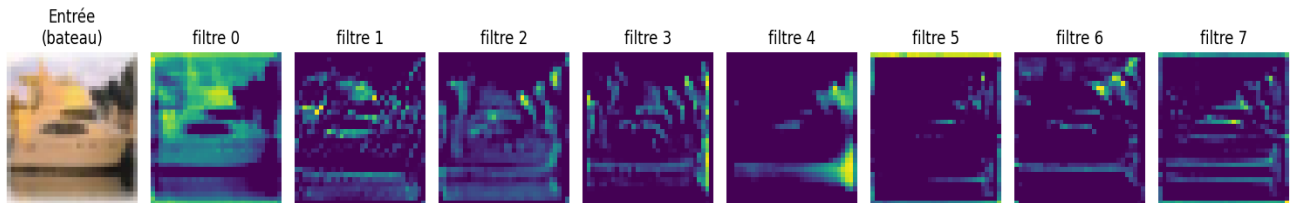


Figure 4 - Cartes de caractéristiques du premier bloc convolucional.

Les premières couches réagissent à des **bords et contrastes locaux**, briques de base des représentations plus abstraites en profondeur.

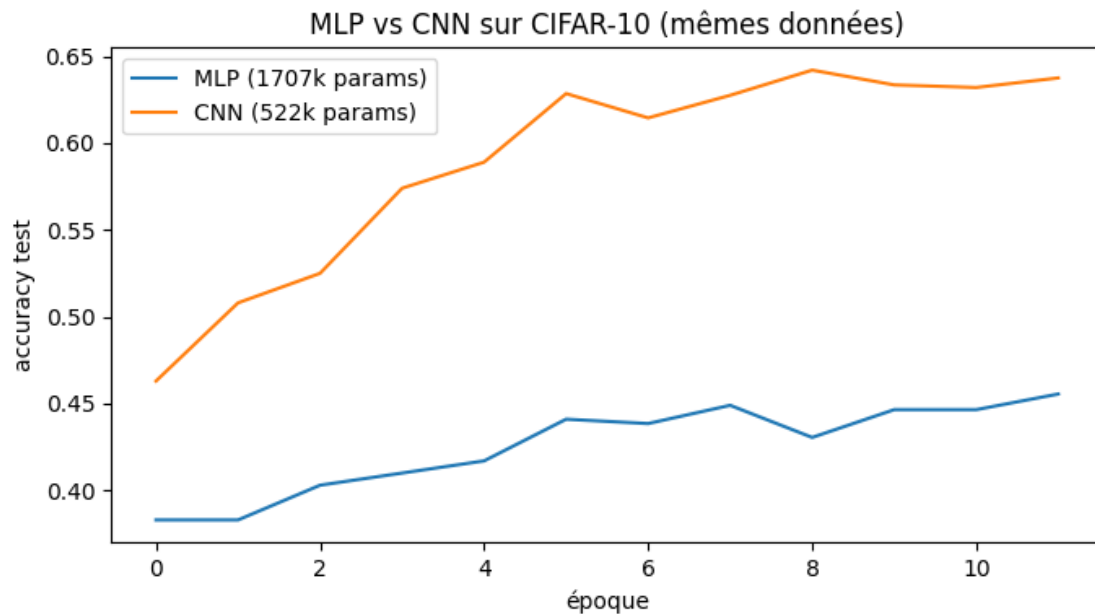


Figure 5 - MLP vs CNN sur les mêmes données.

Résultat central:

Modèle	Accuracy test
MLP (image aplatie)	0,456
CNN (LeNet amélioré)	0,638

Table 3 - Comparaison MLP vs CNN sur CIFAR-10 (accuracy de test, mêmes données et mêmes époques)

Le CNN **domine nettement** le MLP à nombre de paramètres comparable : son biais inductif encode directement la structure des images.

5.4 Question de synthèse — Partie II

Pourquoi un CNN est-il plus pertinent qu'un MLP pour la classification d'images, et comment les choix de padding, stride, pooling et profondeur influencent-ils les performances ?

Le CNN est plus pertinent car son architecture **épouse la structure des images** : localité et invariance par translation, exploitées par le partage des poids (d'où moins de paramètres et une meilleure généralisation), et hiérarchie de représentations (visible sur les cartes de caractéristiques). Les hyperparamètres agissent via la **géométrie du signal** ($H_{out} = \lfloor (H + 2p - k)/s \rfloor + 1$) : le padding préserve la résolution ; le stride sous-échantillonne (gain de calcul, perte de détail) ; le pooling contrôle l'invariance locale (max = saillance, avg = lissage) ; profondeur et nombre de filtres déterminent la capacité. Les ablations confirment que ces choix déplacent réellement l'accuracy.

6. Partie III — RNN, LSTM, GRU et Seq2Seq (Tatoeba fra-eng)

6.1 Cadre théorique

Un **modèle de langage** attribue une probabilité à une séquence et la factorise par la **règle de chaîne** : $P(x_1, \dots, x_T) = \prod_t P(x_t | x_{1:t-1})$. La qualité se mesure par la **perplexité** $PPL = \exp(-\frac{1}{T} \sum \log P(x_t | x_{1:t-1}))$, interprétable comme le nombre moyen de choix équiprobables hésités par le modèle (plus bas = mieux). Un **RNN** résume le passé dans un **état caché** h_t transmis de pas en pas. Son entraînement par **rétropropagation à travers le temps (BPTT)** souffre de gradients **évanescents/explosifs** ; le **gradient clipping** renormalise le gradient au-delà d'un seuil. Les **LSTM** et **GRU** introduisent des **portes** qui créent un chemin stable pour l'information.

6.2 Préparation des données

Pipeline standard de traduction : nettoyage → tokenisation → vocabulaire (avec tokens spéciaux <pad> <bos> <eos> <unk> et fréquence minimale) → padding/troncature → mini-lots. Les positions de padding sont **masquées** dans la perte (entropie croisée masquée). La **phrase source est inversée** (astuce de Sutskever et al.) pour faciliter l'apprentissage des dépendances courtes.

6.3 Comparaison RNN / LSTM / GRU et gradient clipping

Les trois cellules sont entraînées comme modèles de langage à conditions identiques.

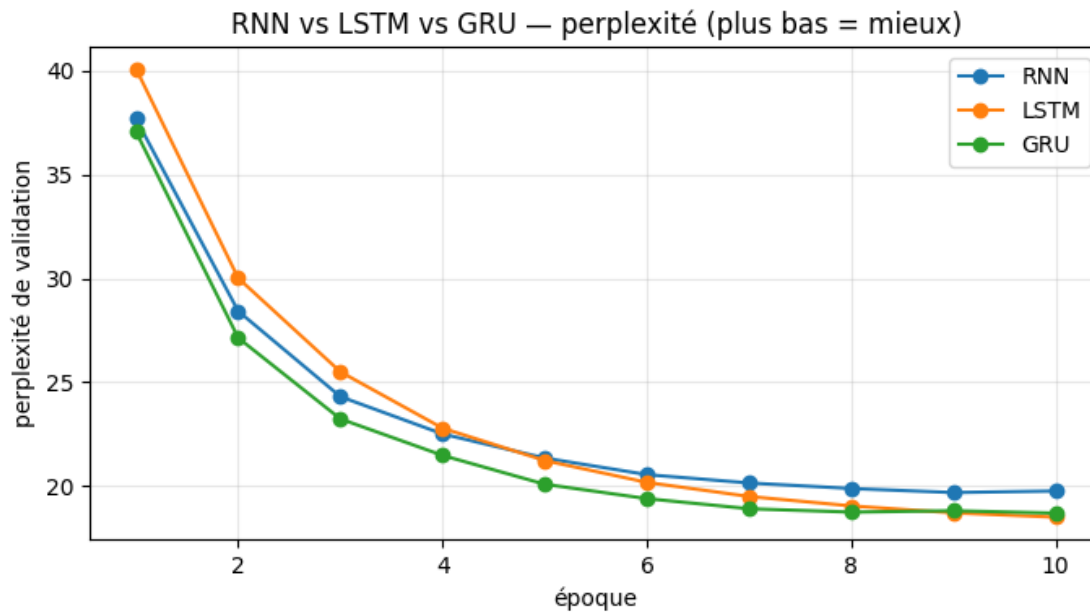


Figure 6 - Perplexité de validation (plus bas = mieux).

Modèle	PPL validation	Paramètres
RNN simple	19,8	615 613
LSTM	18,5	862 909
GRU	18,7	780 477

Table 4 - Perplexité de validation (plus bas = mieux).

Le RNN simple atteint la perplexité la plus élevée ; LSTM et GRU, grâce à leurs portes, font mieux, le **GRU** offrant le meilleur compromis performance/coût — d'où son emploi dans le Seq2Seq.

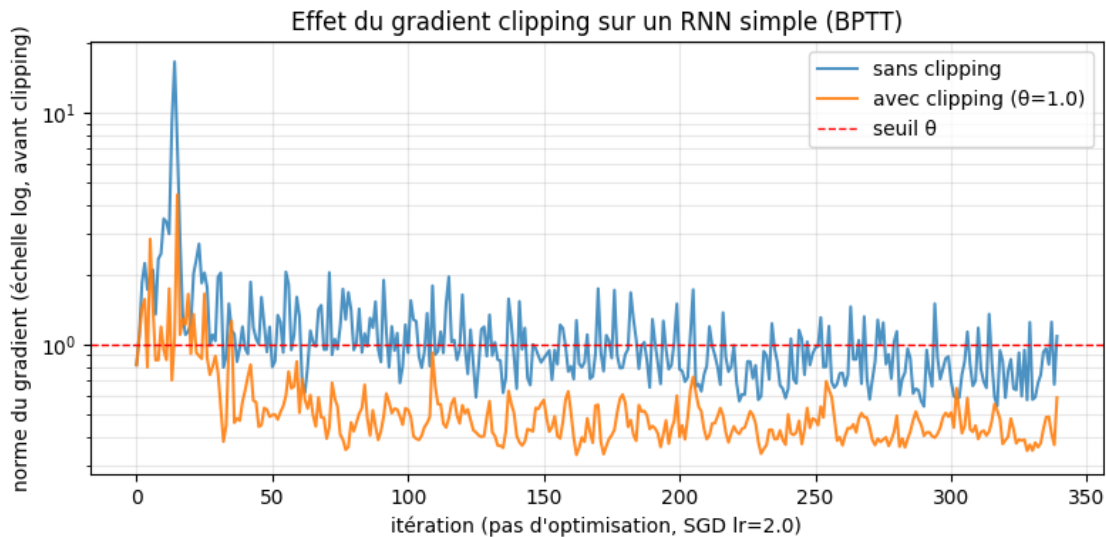


Figure 7 - Effet du gradient clipping (RNN simple, SGD, échelle log).

Sans clipping, la norme du gradient présente des **pics** (max $\approx 16,6$) caractéristiques de l'instabilité de la BPTT ; avec clipping ($\theta = 1$), elle reste **bornée** (max $\approx 4,4$) et l'entraînement se stabilise.

6.4 Système Seq2Seq

Architecture **encodeur-décodeur GRU** modélisant $P(y_{1:T_y} | x_{1:T_x})$. L'encodeur résume la source dans son état final ; le décodeur génère la cible token par token, **conditionné à chaque pas sur le contexte encodeur** (état final, **maintenu fixe** durant tout le décodage — point crucial pour la cohérence entraînement/inférence). L'entraînement utilise le **teacher forcing** et une **perte masquée**. La perplexité de validation finale atteint $\approx 11,5$.

6.5 Décodage et évaluation

Deux stratégies de décodage sont implémentées : **glouton** (argmax à chaque pas) et **beam search** (k hypothèses, score = somme des log-probabilités, **normalisation par la longueur**). La métrique **BLEU** est réimplémentée à la main (précision des n-grammes + pénalité de brièveté), nltk n'étant pas disponible.

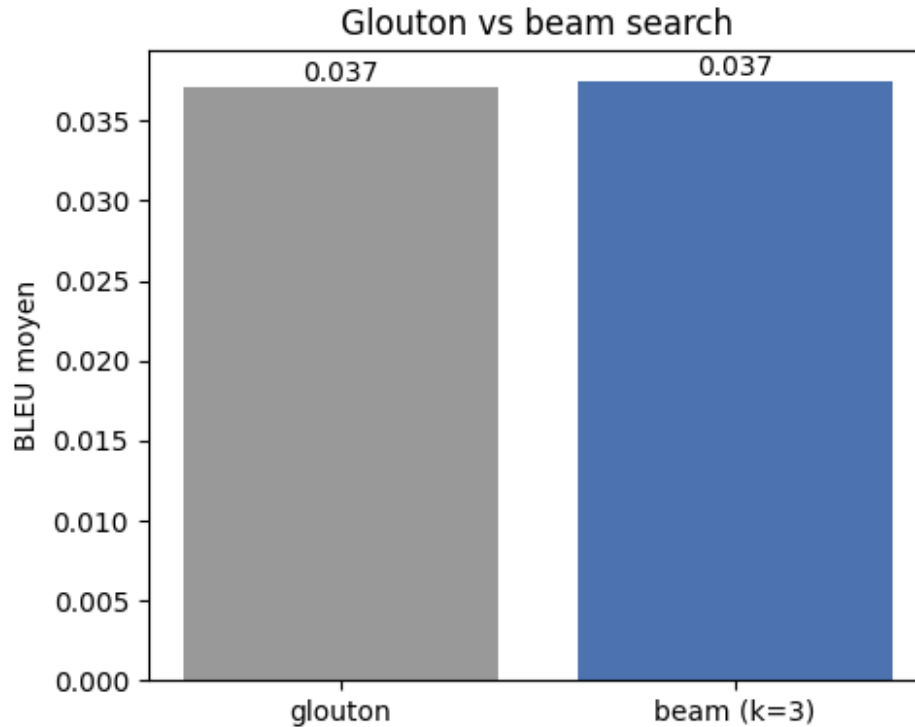


Figure 8 - BLEU moyen sur la validation.

Décodage	BLEU moyen
Glouton	0,0371
Beam (k = 3)	0,0375

Table 5 - Comparaison des stratégies de décodage du Seq2Seq : glouton vs beam search (BLEU moyen sur validation)

Exemples qualitatifs (validation) : the book is easy . → « le livre est facile . » (parfait) ; i don't have a weapon . → « je n'ai pas un crayon . » (grammatical) ; are we disturbing you ? → « est-ce que tu <unk> ? » (structure correcte). Les <unk> correspondent aux mots rares écartés du vocabulaire.

6.6 Question de synthèse — Partie III

Dans quelle mesure les architectures récurrentes modélisent-elles efficacement une séquence, et comment justifier le passage RNN → LSTM/GRU → encodeur–décodeur ?

Les RNN modélisent une séquence en factorisant sa probabilité et en résumant le passé dans un état caché, ce qui gère les **longueurs variables** et l'**ordre**, contrairement à un MLP. Mais le RNN simple est limité par la BPTT (gradients instables, perplexité élevée — observés). Le passage au **LSTM/GRU** se justifie par leurs **portes**, qui stabilisent l'information et améliorent la mémoire longue. Pour la **traduction**, une seule séquence ne suffit pas : le schéma **encodeur–décodeur** conditionne la génération cible sur la source (teacher forcing, perte masquée), puis le **beam search** améliore l'inférence. La progression RNN → LSTM/GRU → Seq2Seq → beam search n'est donc pas arbitraire : chaque étape lève une limite concrète de la précédente.

7. Synthèse transversale

Comment le deep learning adapte-t-il ses architectures à la structure des données — tabulaire, image et séquentielle — et pourquoi un même paradigme supervisé doit-il être décliné différemment ?

Le deep learning repose sur un **noyau commun** (modèle différentiable optimisé par descente de gradient), mais sa puissance vient de l'**injection de biais inductifs adaptés à la structure des données**.

Aspect	Tabulaire (MLP)	Image (CNN)	Séquence (RNN/Seq2Seq)
Structure	vecteur, sans géométrie ni ordre	grille 2D, localité + invariance	suite ordonnée, longueur variable
Biais inductif	aucun a priori	localité, partage des poids, hiérarchie	récence, mémoire, conditionnement
Métrique	accuracy / F1	accuracy	perplexité / BLEU
Résultat clé	acc $\approx$ 0,73	CNN 0,64 $\gg$ MLP 0,46	LSTM/GRU < RNN ; beam $\geq$ glouton

Table 6 - Synthèse transversale : adéquation entre architecture et structure des données

Le constat central est démontré expérimentalement en Partie II : appliquer la « mauvaise » architecture (un MLP sur des images) **effondre la performance** à paramètres comparables. Un même paradigme supervisé doit donc être décliné différemment parce que la **structure statistique** des données — géométrie, dépendance locale, temporalité, représentation — détermine quelles invariances et contraintes rendent l'apprentissage **efficace en données** et **généralisable**. Le rôle de l'ingénieur est de **diagnostiquer la structure des données** puis de **choisir l'architecture dont le biais inductif y correspond**.

Ouverture. L'**attention** et les **Transformers** prolongent cette logique en remplaçant le biais de localité/récence par un biais d'**alignement appris**, levant le goulot d'étranglement du contexte unique observé en Partie III.

8. Limites

- **Exécution CPU + sous-échantillonnage** : les performances absolues (accuracy/BLEU) sont en deçà de l'état de l'art ; seules les **tendances comparatives** sont revendiquées, mais elles sont obtenues à conditions contrôlées et **reproductibles** en augmentant les paramètres de CONFIG.
- **Partie I** : petit jeu, cible bruitée ; une validation croisée et une comparaison à un Gradient Boosting renforceraient l'analyse.
- **Partie II** : peu d'époques ; réseaux plus profonds et augmentation de données amélioreraient les scores.
- **Partie III** : vocabulaire et corpus réduits ; un **mécanisme d'attention** lèverait la limite du contexte unique.

9. Conclusion

Ce projet a couvert, de bout en bout et sous PyTorch, les trois grandes familles d'architectures du module : MLP (tabulaire), CNN (image) et RNN/LSTM/GRU/Seq2Seq (séquence). Pour chacune, l'étude théorique, l'implémentation rigoureuse (deux versions du MLP, opérations convolutionnelles réimplémentées et validées contre PyTorch, comparaison des cellules récurrentes, système Seq2Seq avec beam search) et les expériences contrôlées ont permis d'**illustrer concrètement** le principe directeur : **l'efficacité d'un modèle de deep learning provient de l'adéquation entre son biais inductif et la structure statistique des données**. Les résultats — CNN $\gg$ MLP sur l'image, LSTM/GRU $>$ RNN sur la séquence, beam $\geq$ glouton au décodage — confirment expérimentalement la théorie et répondent aux questions de synthèse de chaque partie ainsi qu'à la problématique transversale.

10. Annexe expérimentale

Toutes les figures sont issues de l'exécution des notebooks (Partie I à Partie III) :

- **Figure 1** : convergence selon l'initialisation (Partie I).
- **Figure 2** : matrice de confusion du MLP (Partie I).
- **Figure 3** : ablations architecturales du CNN (Partie II).
- **Figure 4** : cartes de caractéristiques (Partie II).
- **Figure 5** : MLP vs CNN (Partie II).
- **Figure 6** : perplexité RNN/LSTM/GRU (Partie III).
- **Figure 7** : effet du gradient clipping (Partie III).
- **Figure 8** : BLEU glouton vs beam (Partie III).

Tableaux comparatifs et métriques détaillées : voir sections 4 à 6. Code source complet et commenté : notebooks Partie1_MLP_WineQuality.ipynb, Partie2_CNN_CIFAR10.ipynb, Partie3_RNN_Seq2Seq_FraEng.ipynb et Partie4_Synthese_Transversale.ipynb.

11. Pourquoi ces choix ? — explications simples et justifications

Cette section explique, dans un langage simple, **pourquoi** chaque modèle, technologie et technique a été choisi. L'idée directrice est toujours la même : **on choisit l'outil dont la « façon de voir » correspond à la nature des données.**

11.1 En une phrase : le bon modèle pour le bon type de données

- **Données en tableau (Wine Quality) → MLP.** Les colonnes (alcool, pH, acidité...) n'ont ni ordre ni voisinage : « alcool » n'est pas « à côté » de « pH ». Un MLP regarde toutes les variables ensemble, sans a priori de position — c'est exactement ce qu'il faut ici.
- **Images (CIFAR-10) → CNN.** Sur une image, les pixels proches forment des motifs (un bord, un œil) et un même motif peut apparaître n'importe où. Le CNN apprend un petit motif une seule fois et le reconnaît partout.
- **Texte (fra-eng) → RNN / LSTM / GRU / Seq2Seq.** Une phrase est une suite ordonnée où chaque mot dépend des précédents. Ces modèles lisent mot à mot en gardant une mémoire du passé.

11.2 Pourquoi un MLP pour les données tabulaires (Partie I)

Un MLP empile des couches qui combinent toutes les variables avec des poids, puis appliquent une non-linéarité (ReLU). **Argument :** comme les variables tabulaires n'ont pas de structure spatiale ou temporelle à exploiter, il est inutile (et contre-productif) d'imposer un a priori comme la convolution. Le MLP, « neutre », est donc le choix naturel et pédagogiquement central pour comprendre nn.Module, les paramètres et la rétropropagation.

Limite assumée : sur ce type de données, des modèles à base d'arbres (Random Forest, Gradient Boosting) sont souvent au moins aussi bons — ce que le rapport discute honnêtement.

11.3 Pourquoi un CNN pour les images (Partie II)

- **Localité.** Un filtre ne regarde qu'un petit carré de pixels à la fois : il capte les motifs locaux (bords, coins, textures).
- **Partage des poids.** Le même filtre glisse sur toute l'image. On apprend donc beaucoup moins de paramètres qu'un MLP, et on reconnaît un motif où qu'il soit (invariance par translation).
- **Hiérarchie.** En empilant les couches, le réseau combine bords → formes → objets.

Preuve dans le projet : à nombre de paramètres comparable, le CNN atteint ~0,64 contre ~0,46 pour le MLP sur les mêmes images. C'est l'argument expérimental le plus fort du projet : la bonne architecture compte plus que la seule taille du modèle.

11.4 Pourquoi RNN, puis LSTM/GRU, puis Seq2Seq (Partie III)

- **RNN simple.** Il lit la phrase mot à mot en gardant une « mémoire » (état caché). Problème : sur les phrases longues, la mémoire s'efface (gradient qui s'évanouit) ou explose — d'où une perplexité plus élevée et de l'instabilité.

- **LSTM / GRU.** Ils ajoutent des « portes » qui décident quoi garder, oublier ou écrire. Résultat : une mémoire plus stable et de meilleurs scores. Le GRU est plus léger que le LSTM pour des performances proches → bon compromis, retenu pour le Seq2Seq.

- **Seq2Seq (encodeur–décodeur).** Traduire, c'est passer d'une phrase source à une phrase cible de longueur différente. Un seul RNN ne suffit pas : on met un encodeur qui « lit » l'anglais et résume le sens, puis un décodeur qui « écrit » le français à partir de ce résumé.

Logique d'ensemble : chaque étape corrige une limite de la précédente (mémoire instable → portes ; un seul RNN insuffisant → encodeur-décodeur ; décodage glouton sous-optimal → beam search).

11.5 Pourquoi PyTorch comme framework

- **Demandé par le sujet.** Le cahier des charges impose explicitement une implémentation sous PyTorch.

- **Simple et lisible.** Le graphe de calcul est dynamique (« define-by-run ») : le code se lit comme du Python normal, idéal pour comprendre et déboguer.

- **Autograd intégré.** La rétropropagation est automatique : on écrit la passe avant, PyTorch calcule les gradients.

- **Écosystème complet.** torchvision fournit directement CIFAR-10 ; les couches nn.RNN / nn.LSTM / nn.GRU / nn.Conv2d existent prêtes à l'emploi, ce qui évite de réinventer la roue (tout en réimplémentant « à la main » les opérations clés pour la compréhension).

11.6 Pourquoi ces techniques (en une ligne chacune)

- **Initialisation de Xavier.** Démarre l'entraînement avec des poids ni trop grands ni trop petits → le signal se propage bien et l'apprentissage est stable (l'initialisation constante échoue car tous les neurones restent identiques).

- **Normalisation des variables (StandardScaler).** Met toutes les variables à la même échelle pour que le modèle ne privilégie pas une variable juste parce que ses nombres sont grands. Ajustée sur le train seulement, pour ne pas « tricher ».

- **BatchNorm.** Stabilise et accélère l'entraînement du CNN en normalisant les activations.

- **Dropout.** Éteint des neurones au hasard pendant l'entraînement → empêche d'apprendre « par cœur » (surapprentissage).

- **Gradient clipping.** Plafonne la taille du gradient pour empêcher les « sauts » trop violents qui font diverger un RNN (montré expérimentalement : pic ~16,6 sans, ~4,4 avec).

- **Teacher forcing.** À l'entraînement, on fournit au décodeur le vrai mot précédent plutôt que sa propre prédiction → convergence plus rapide et stable.

- **Masquage (masking).** On ignore les positions de « remplissage » (padding) dans le calcul de la perte, pour ne pas fausser l'apprentissage avec des cases vides.

- **Décodage glouton vs beam search.** Le glouton prend le meilleur mot à chaque pas (rapide mais myope) ; le beam search garde plusieurs pistes en parallèle → de meilleures phrases.
- **Perplexité.** Mesure à quel point le modèle est « surpris » par le texte ; plus elle est basse, mieux c'est.
- **BLEU.** Compare la traduction produite à une référence via les n-grammes communs ; réimplémentée à la main car la librairie nltk n'est pas disponible.

11.7 Pourquoi ces choix de données et d'exécution

- **Mirrors publics au lieu de Kaggle.** Les datasets Kaggle exigent un compte et une clé d'API ; les sources UCI (Wine), torchvision (CIFAR-10) et le mirror d2l (fra-eng) fournissent exactement les mêmes données sans authentification → les notebooks se téléchargent et s'exécutent partout, de façon reproductible.
- **Sous-échantillonnage et entraînement court.** La machine est en CPU uniquement (pas de GPU). On réduit la taille des données et le nombre d'époques pour que tout tourne en quelques minutes. Les conclusions comparatives restent valables, et tout est paramétrable (CONFIG) pour passer à pleine échelle sur GPU.